EECS 348 Assignment 4 - GenAI Code Comparison

1. AI systems and access

Claude: accessed through the Claude chat interface; code supplied by the student. Google

Gemini:accessed through the Gemini chat interface; code supplied by the student. The same prompt was requested for both. The full prompt used in the conversation is reproduced below.

2. Identical prompt

I need you to generate a complete C++ program that solves Sudoku puzzles using object-oriented programming and recursive backtracking.

1. Create a C++ program named Assignment4.cpp.
2. Use object-oriented programming with a class named SudokuSolver that contains the puzzle grid and methods for solving it.
3. Implement a recursive depth-first search with backtracking to solve Sudoku puzzles.
4. Read five input files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
5. Each input file contains a 9×9 Sudoku grid with digits 1–9 separated by spaces. Empty cells are represented by underscores (_).
6. Follow standard Sudoku rules:
    - Each row must contain digits 1–9 exactly once.
    - Each column must contain digits 1–9 exactly once.
    - Each 3×3 subgrid must contain digits 1–9 exactly once.
7. For each puzzle, print:
    - The puzzle filename.
    - The original puzzle as read from the file.
    - The completed Sudoku solution.
    - If no solution exists, print "No solution found".
8. If a puzzle has multiple valid solutions, print every solution.
9. Validate the initial puzzle to detect conflicting numbers.
10. Use standard C++ libraries and ensure the program compiles with g++ on a Linux server using C++17.
11. Use classes and member methods to organize the program rather than relying on standalone functions.
12. Include a prologue comment containing the program name, description, inputs, outputs, author placeholder, creation date, and GenAI attribution.
13. Include descriptive comments explaining the major blocks of code and individual operations.
14. Use readable variable names, consistent formatting, and good software engineering practices.

3. Correctness and test results

Both source programs compiled locally with g++ -std=c++17 -O2 -Wall -Wextra -pedantic. Claude compiled without warnings Gemini compiled with a member initialization-order warning. Both returned the same counts on the five supplied files: puzzle1=1, puzzle2=1, puzzle3=1, puzzle4=0 (No solution found), puzzle5=2. Both print original puzzles and enumerate all found solutions. This verifies these five inputs, not every possible malformed input.

4. Execution time and algorithmic complexity

Claude uses bit masks for constant-time candidate membership checks and selects the empty cell with the fewest candidates (MRV), reducing the number of failed recursive branches in typical puzzles. Gemini uses fixed row-major traversal and scans the row, column and box for each proposed digit. Both algorithms are exponential in the number k of blanks in the worst case (roughly O(9^k) search branches, with constant work per node for a fixed 9x9 board). MRV improves practical search efficiency, not the worst-case exponential bound. No timing benchmark is known

5. Space complexity and maintainability

Claude uses fixed-size arrays and bitmasks but stores every completed solution in a vector, using O(81S + k) memory for S solutions, with recursion depth O(k). Gemini prints solutions as they are discovered, avoiding O(S) solution storage; it uses a 9x9 dynamic grid and O(81) recursion depth including filled positions. Gemini also copies its whole SudokuSolver for each given cell during validation, creating unnecessary allocation and work. Claude separates loading, validation, solving and presentation and gives specific diagnostics. Gemini is shorter and easier to follow initially, but has weaker input-length validation and a compiler warning.

6. Selection and justification

Claude was selected as the stronger starting point because it uses MRV and bitmasks, provides strict 81-token parsing and conflict diagnostics, and compiled without warnings. Both correctly solved the supplied puzzle set, so the choice is based on algorithm design, robustness and maintainability rather than a claimed measured speedup.

7. Improvements in final Assignment4.cpp

The final implementation is a reviewed revision not a verbatim copy of Claude. It retains recursive exhaustive search with a most constrained cell strategy, uses fixed size grid storage, validates input, prints the original puzzle, and enumerates all solutions. Unlike Claude, the final code prints each solution as found instead of keeping all solutions in memory, reducing solution storage cost from O(S) grids to constant output buffering its recursion stack remains O(k). Named methods and detailed comments separate responsibilities.

8. Complete raw Claude code

Code is in Github Folder

9. Complete raw Gemini code

Code is in Github Folder